
Sorting Algorithms

<i>conducted by:</i> Adhm Mohamed ElShabasy 23010270

Contents

- Code Structure & Design
- Algorithm Implementations
- The Comparison Tab
- The Visualization Tab
- The Dual Visualization Tab
- Sample Outputs
- Complexity Summary
- Comparison Results & Analysis

1 Code Structure and Design

1.1 Class Hierarchy

The project follows an object-oriented design centered on a common abstract base class.

```
Sort                (abstract base)
  SelectionSort
  InsertionSort
  BubbleSort
  MergeSort
  HeapSort
  QuickSort

Generator           (array generation utility)
InputReader         (file input reader)
MainApp             (JavaFX application)
  ComparisonResult  (data model for the table)
```

1.2 The Sort Abstract Class

All sorting algorithms extend the `Sort` abstract class, which defines the shared interface and state:

```
1 public abstract class Sort {
2     public long interchanges;
3     public long comparisons;
4
5     public abstract void sort(int[] array);
6
7
8
9     public void reset() {
10         interchanges = 0;
11         comparisons = 0;
12     }
13
14     public void swap(int[] array, int i, int j) {
15         int temp = array[i];
16         array[i] = array[j];
17         array[j] = temp;
18     }
19 }
```

Design decisions:

- `comparisons` and `interchanges` are instance fields, not static, so each algorithm instance maintains its own counters independently. This is critical for the multi-threaded comparison tab.
- `swap()` is a concrete method on the base class. This was a key enabler for the visualization system: the visualizer overrides `swap()` in anonymous subclasses to intercept every swap and record a snapshot.
- `reset()` allows the same sorter instance to be reused across multiple runs without creating new objects.

1.3 The Generator Class

Generates input arrays in three modes controlled by an integer `op`:

- `op=0`: Random integers in range `[0, bound]`
- `op=1`: Already sorted (ascending sequence `0, 1, 2, ...`)
- `op=2`: Inversely sorted (descending sequence `n, n-1, ..., 1`)

1.4 The InputReader Class

Reads integer arrays from text files. The reader is designed to be format-tolerant:

- Strips any bracket, brace, or parenthesis characters (`[ ] { } ( )`)
- Splits on both commas and whitespace, so files formatted as `1,2,3` or `1 2 3` or mixed are all accepted

2 Algorithm Implementations

2.1 Selection Sort

Algorithm 1 Selection Sort

```
1: for  $i \leftarrow 0$  to  $n - 1$  do
2:    $minIndex \leftarrow i$ 
3:   for  $j \leftarrow i + 1$  to  $n - 1$  do
4:     compare  $array[j]$  with  $array[minIndex]$ 
5:     if  $array[j] < array[minIndex]$  then
6:        $minIndex \leftarrow j$ 
7:     end if
8:   end for
9:   if  $minIndex \neq i$  then
10:    swap  $array[i]$  and  $array[minIndex]$ 
11:  end if
12: end for
```

Complexity: $O(n^2)$ comparisons in all cases. $O(n)$ swaps at most. Best suited for small arrays where write operations are expensive.

2.2 Insertion Sort

Algorithm 2 Insertion Sort

```
1: for  $i \leftarrow 1$  to  $n - 1$  do
2:    $temp \leftarrow i$ 
3:   for  $j \leftarrow temp - 1$  downto 0 do
4:     compare  $array[j]$  with  $array[temp]$ 
5:     if  $array[j] > array[temp]$  then
6:       swap  $array[j]$  and  $array[temp]$ 
7:        $temp \leftarrow temp - 1$ 
8:     else
9:       break
10:    end if
11:  end for
12: end for
```

Complexity: $O(n^2)$ worst case, $O(n)$ best case (already sorted). The inner loop breaks early when the correct position is found, making it efficient on nearly-sorted data.

2.3 Bubble Sort

Algorithm 3 Bubble Sort

```
1: for  $i \leftarrow 0$  to  $n - 2$  do
2:   for  $j \leftarrow 0$  to  $n - 2$  do
3:     compare  $array[j]$  with  $array[j + 1]$ 
4:     if  $array[j] > array[j + 1]$  then
5:       swap  $array[j]$  and  $array[j + 1]$ 
6:     end if
7:   end for
8: end for
```

Complexity: $O(n^2)$ in all cases. The implementation uses a fixed outer loop and does not include an early-exit optimisation, so it always performs the full n^2 comparisons.

2.4 Merge Sort

Algorithm 4 Merge Sort

```
1: procedure MERGESORT(array)
2:   if length = 1 then return
3:   end if
4:   Split array into left and right halves
5:   MERGESORT(left)
6:   MERGESORT(right)
7:   MERGE(array, left, right)
8: end procedure
9: procedure MERGE(array, left, right)
10:   $i \leftarrow 0, j \leftarrow 0, k \leftarrow 0$ 
11:  while  $i < |left|$  and  $j < |right|$  do
12:    compare left[i] with right[j]
13:    if left[i] ≤ right[j] then
14:      array[k ++] ← left[i ++]
15:    else
16:      array[k ++] ← right[j ++]
17:    end if
18:  end while
19:  Copy remaining elements of left and right into array
20: end procedure
```

Complexity: $O(n \log n)$ in all cases. Uses $O(n)$ extra space for the temporary arrays during merging.

2.5 Heap Sort

Algorithm 5 Heap Sort

```

1: procedure HEAPSORT(array)
2:   BUILDMAXHEAP(array)
3:   end  $\leftarrow n - 1$ 
4:   while end  $\geq 0$  do
5:     swap array[0] and array[end]
6:     end  $\leftarrow end - 1$ 
7:     MAXHEAPIFY(0, array)
8:   end while
9: end procedure
10: procedure MAXHEAPIFY(i, array)
11:   l  $\leftarrow 2i + 1$ , r  $\leftarrow 2i + 2$ , largest  $\leftarrow i$ 
12:   if r < heapLength and array[r] > array[i] then largest  $\leftarrow r$ 
13:   end if
14:   if l < heapLength and array[l] > array[largest] then largest  $\leftarrow l$ 
15:   end if
16:   if largest  $\neq i$  then
17:     swap array[i] and array[largest]
18:     MAXHEAPIFY(largest, array)
19:   end if
20: end procedure

```

Complexity: $O(n \log n)$ in all cases. Sorts in-place with $O(1)$ extra space.

2.6 Quick Sort

Algorithm 6 Quick Sort

```

1: procedure QUICKSORT(array, l, h)
2:   if  $l \geq h$  then return
3:   end if
4:    $pivotIndex \leftarrow$  middle of  $[l, h]$ 
5:    $p \leftarrow array[pivotIndex]$ 
6:   swap  $array[pivotIndex]$  and  $array[l]$ 
7:    $i \leftarrow l$ ,  $j \leftarrow l + 1$ 
8:   while  $j \leq h$  do
9:     compare  $array[j]$  with  $p$ 
10:    if  $array[j] \geq p$  then
11:       $j \leftarrow j + 1$ 
12:    else
13:       $i \leftarrow i + 1$ 
14:      swap  $array[i]$  and  $array[j]$ 
15:       $j \leftarrow j + 1$ 
16:    end if
17:  end while
18:  swap  $array[i]$  and  $array[l]$ 
19:  QUICKSORT(array, l,  $i - 1$ )
20:  QUICKSORT(array,  $i + 1$ , h)
21: end procedure

```

Complexity: $O(n \log n)$ average, $O(n^2)$ worst case. The visualizer uses the middle element as pivot (rather than random) to avoid worst-case behaviour on sorted input during animation.

3 The Comparison Tab

3.1 Design

The comparison tab allows the user to benchmark any combination of the six algorithms against each other on the same input. Settings include:

- **Input type:** Random, Sorted, Inversely Sorted, or from File
- **Array size:** Up to 10,000 elements
- **Bound:** Upper bound for random number generation
- **Runs:** Number of repeated runs to average over
- **Multiple files:** When using the file input mode, multiple files can be selected at once. Each file is treated as one run, and the runs field auto-updates to match the number of files selected.

Results are displayed in a table showing average, minimum, and maximum execution time (in milliseconds), plus average comparisons and interchanges across all runs.

3.2 Threading Design

Each selected algorithm is dispatched as a separate `javafx.concurrent.Task` into a `FixedThreadPool`. This allows all selected algorithms to run concurrently rather than sequentially.

```
1 ExecutorService pool =
2     Executors.newFixedThreadPool(Math.min(total, 6));
3
4 for (String algoName : selected) {
5     Task<ComparisonResult> task = new Task<>() {
6         @Override protected ComparisonResult call() {
7             // sort and measure time
8         }
9     };
10    task.setOnSucceeded(e -> Platform.runLater(() -> {
11        comparisonData.add(task.getValue());
12    }));
13    pool.submit(task);
14 }
```

Design decisions:

- The pool size is capped at 6 (one per algorithm) to prevent excessive thread creation.
- `Platform.runLater()` is used to update the UI from the background threads, which is required by JavaFX's single-threaded rendering model.
- Each task gets its own `Sort` instance and its own copy of the base array via `Arrays.copyOf()`, so there is no shared mutable state between threads.
- A shared `done[]` counter (a one-element array used as a mutable integer inside a lambda) tracks completion and re-enables the Run button when all tasks finish.

4 The Visualization Tab

4.1 Design Overview

The visualization tab animates the sorting process by replaying a pre-recorded sequence of array snapshots, one frame at a time, on a JavaFX `Canvas`.

4.2 Step Recording Technique

The visualization does not run the sort live on a background thread. Instead, it runs the sort synchronously before playback begins and records every significant event as a snapshot. This approach has several advantages:

- Playback speed can be changed freely without affecting the sort logic.
- Stepping backwards is trivial since all states are already stored.
- No synchronisation primitives are needed between the sort and the UI.

Three parallel lists are maintained, all indexed by step number:

List	Content
vizSteps	Full copy of the array at this step
highlightPairs	<code>int[] {i, j}</code> indicating which two bars to highlight
isCompStep	true if this is a comparison, false if a swap/write

4.3 The Step-Capturing Sorter

For each algorithm, an anonymous subclass is created inside `buildCapturingSorter()`. For swap-based algorithms (Selection, Insertion, Bubble, Heap, Quick), `swap()` is overridden to record a snapshot after every swap. The sort logic is also overridden to record a *comparison step* (yellow highlight) before every conditional comparison.

For Merge Sort, since it does not use `swap()` at all, the entire merge logic is reimplemented to operate on index ranges within the full array, as it had problems running in the visualizer as it used to show every subarray on its own not the whole thing, recording both comparison steps and write steps:

```

1 case "Merge Sort" -> new Sort() {
2     int[] full;
3     @Override public void sort(int[] array) {
4         full = array;
5         mergeSort(0, array.length - 1);
6     }
7     void merge(int lo, int mid, int hi) {
8         int[] left = Arrays.copyOfRange(full, lo, mid + 1);
9         int[] right = Arrays.copyOfRange(full, mid + 1, hi + 1);
10        int i = 0, j = 0, k = lo;
11        while (i < left.length && j < right.length) {
12            // record comparison step (yellow)
13            steps.add(Arrays.copyOf(full, full.length));
14            pairs.add(new int[] {lo+i, mid+1+j});
15            compFlags.add(true);
16            if (left[i] <= right[j]) full[k++] = left[i++];
17            else full[k++] = right[j++];
18            // record write step (red)
19            steps.add(Arrays.copyOf(full, full.length));
20            pairs.add(new int[] {k-1, k-1});
21            compFlags.add(false);
22        }
23        // ... drain remaining elements
24    }
25 };

```

Crucially, the snapshot is always `Arrays.copyOf(full, full.length)` – a copy of the entire array – so every frame always shows all n bars, never just a subarray.

4.4 Rendering the Bar Chart

The bar chart is drawn on a JavaFX Canvas using `GraphicsContext`. The canvas is bound to its parent pane so it resizes automatically with the window:

```

1 canvas.widthProperty().bind(canvasPane.widthProperty());
2 canvas.heightProperty().bind(canvasPane.heightProperty());

```

The drawing logic in `drawOnCanvas()` works as follows:

1. Clear the canvas with a dark background colour (`#1e1e2e`).

2. Compute bar width as `canvasWidth / n`.
3. For each bar i , compute its height proportionally: $barHeight = \frac{array[i]}{max} \times (canvasHeight - 4)$.
4. Colour bars using HSB colour space, mapping value to hue between 200° (blue) and 340° (purple), creating a gradient effect based on value.
5. Highlighted bars use fixed colours: **yellow** (`#f1fa8c`) for comparisons, **red** (`#ff5555`) for swaps/writes.
6. A 1px padding is applied between bars when bar width exceeds 4px.

4.5 Handling Negative Numbers and Zero-Height Bars

When loading data from a file, values may be zero or negative, which would produce invisible bars. The array is shifted before recording steps using the formula:

$$shifted_value_i = value_i - \min(array) + 1$$

This guarantees the minimum bar height is always 1, while preserving the relative order of all elements (and therefore the correctness of the sort).

4.6 Animation with AnimationTimer

Playback is driven by a JavaFX `AnimationTimer`, which calls its `handle(long now)` method on every rendered frame (typically 60 fps). The timer computes how many steps to advance based on elapsed time and the speed slider:

```

1 animTimer = new AnimationTimer() {
2     @Override public void handle(long now) {
3         long elapsed = now - lastFrameNanos;
4         int steps = (int)(elapsed / intervalNanos);
5         if (steps < 1) return;
6         lastFrameNanos += steps * intervalNanos;
7         for (int i = 0; i < steps; i++) {
8             if (vizStepIndex >= vizSteps.size()-1) {
9                 stopPlay(); return;
10            }
11            vizStepIndex++;
12        }
13        redraw();
14        updateVizLabels();
15    }
16 };

```

`intervalNanos` is derived from the speed slider value (1–500 ms per step). When playback reaches the last step, `stopPlay()` is called automatically.

4.7 Sound

Each swap/write step optionally plays a short sine-wave beep whose frequency is proportional to the value of the highlighted bar:

$$freq = 200 + \frac{value}{max} \times 800 \quad (\text{Hz})$$

Audio is rendered on a dedicated daemon thread using a `SourceDataLine` with a small buffer (2048 bytes) to minimise latency. When playback stops, `audioLine.flush()` is called to immediately discard any buffered audio, preventing sound from continuing after the animation stops.

4.8 Live Counters

Comparisons and interchanges are not simply taken from the final sorter state – they are computed live from the step lists up to the current index:

```

1 long comps = 0, swaps = 0;
2 for (int i = 0; i <= vizStepIndex; i++) {
3     if (isCompStep.get(i)) comps++;
4     else if (highlightPairs.get(i) != null) swaps++;
5 }

```

This ensures the counters tick upward in real time as the animation progresses, and tick back down when stepping backward.

5 The Dual Visualization Tab

The dual visualization tab runs two algorithms side-by-side on the same input array, allowing direct visual comparison. Both algorithms receive an identical copy of the same base array. Steps are recorded independently for each algorithm, and both `AnimationTimer` instances are driven by a single timer that advances both step indices on every frame. When both animations reach their last step, playback stops.

6 Sample Outputs

6.1 Sorting Comparison Tab

Algorithm	Array Size	Input Type	Runs	Avg Time (ms)	Min Time (ms)	Max Time (ms)	Comparisons	Interchanges
Heap Sort	1000	Random	5	0.0903	0.0759	0.1251	19126	9062
Quick Sort	1000	Random	5	0.0780	0.0744	0.0884	10945	5586
Selection Sort	1000	Random	5	0.3476	0.2807	0.5399	500500	990
Merge Sort	1000	Random	5	0.4444	0.2011	0.9172	8680	9976
Insertion Sort	1000	Random	5	0.6030	0.4723	0.7644	499500	247561
Bubble Sort	1000	Random	5	1.2794	1.1018	1.7121	998001	241238
Quick Sort	1000	Sorted	20	0.0530	0.0474	0.0731	11057	6016
Heap Sort	1000	Sorted	20	0.0730	0.0588	0.1344	20420	9709
Selection Sort	1000	Sorted	20	0.3054	0.2477	0.4080	500500	0
Merge Sort	1000	Sorted	20	0.4290	0.1579	1.4668	5044	9976
Insertion Sort	1000	Sorted	20	0.5113	0.3257	0.7784	499500	0
Bubble Sort	1000	Sorted	20	0.7063	0.5885	1.0445	998001	0
Quick Sort	5400	Inversely Sorted	47	0.3308	0.2063	0.4405	79297	41786
Heap Sort	5400	Inversely Sorted	47	0.3800	0.2778	0.4976	122044	58321
Merge Sort	5400	Inversely Sorted	47	1.8861	0.8668	5.2792	32380	67408
Selection Sort	5400	Inversely Sorted	47	7.1919	6.4398	12.8793	14582700	2700
Insertion Sort	5400	Inversely Sorted	47	17.6470	16.4905	28.7330	14577300	14577300
Bubble Sort	5400	Inversely Sorted	47	25.9144	23.0190	52.9012	29149201	14577300

Figure 1: Sorting Comparison tab showing benchmark results

6.2 Sorting Visualization Tab

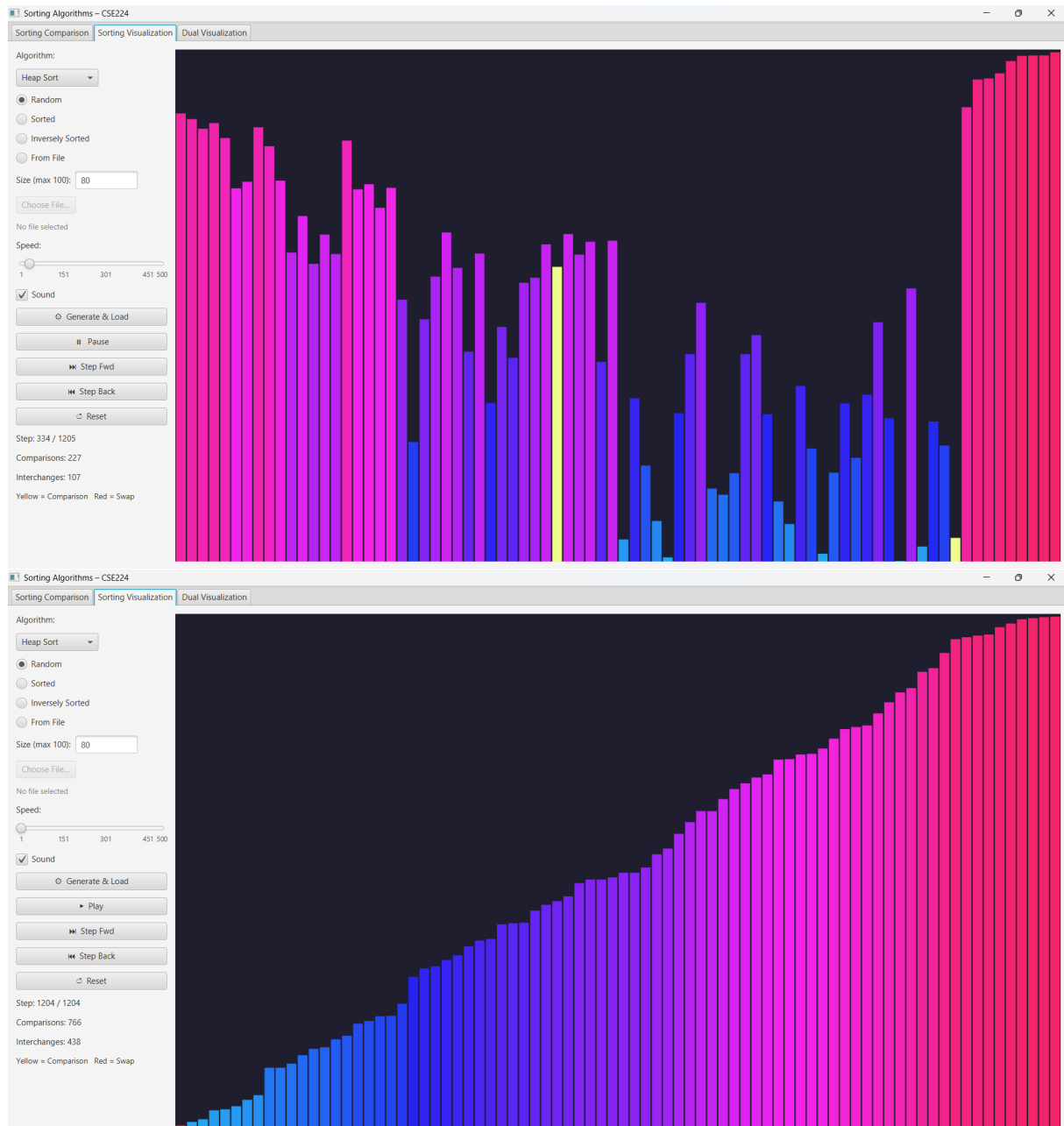


Figure 2: Sorting Visualization tab mid-animation

6.3 Dual Visualization Tab

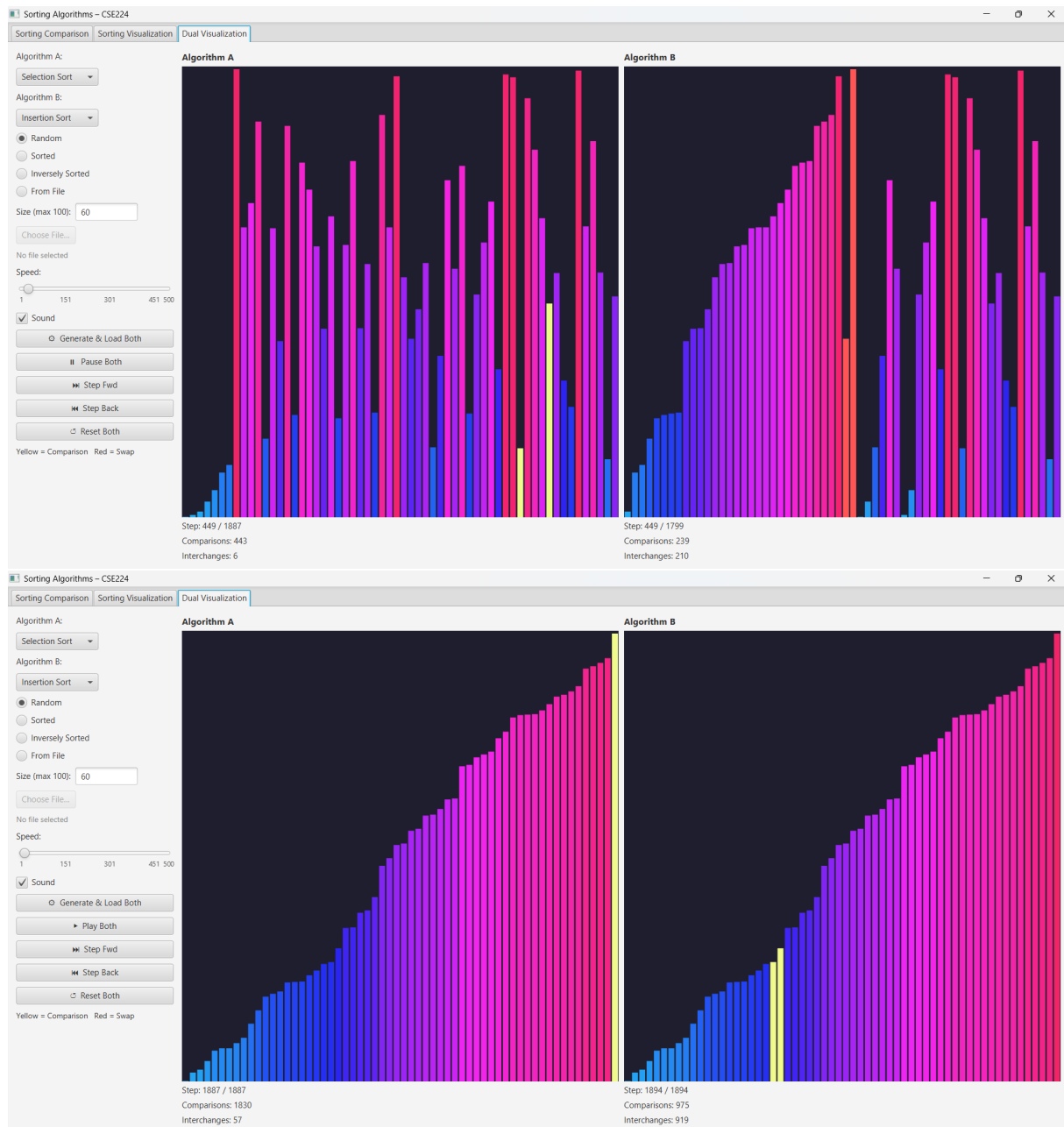


Figure 3: Dual Visualization tab comparing two algorithms

7 Complexity Summary

Algorithm	Best	Average	Worst	Space
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

Table 1: Time and space complexity of all implemented algorithms

8 Comparison Results & Analysis

8.1 Raw Data Tables

Table 2: Benchmark Results – Random Input

Algorithm	Size	Avg (ms)	Min (ms)	Max (ms)	Comparisons	Interchanges
Quick Sort	100	0.0940	0.0875	0.1078	648	344
Selection Sort	100	0.1537	0.1414	0.1944	5050	97
Insertion Sort	100	0.2464	0.1993	0.2728	4950	2494
Bubble Sort	100	0.5191	0.4077	0.7204	9801	2436
Heap Sort	100	0.1256	0.1031	0.1607	1278	588
Merge Sort	100	0.0896	0.0860	0.0933	528	672
Merge Sort	1000	0.2861	0.1793	0.6522	8687	9976
Quick Sort	1000	0.2992	0.1517	0.8079	10940	5823
Heap Sort	1000	0.3407	0.2637	0.4261	19076	9037
Selection Sort	1000	3.4540	2.2558	6.1181	500500	993
Insertion Sort	1000	3.5460	1.4703	5.7982	499500	255186
Bubble Sort	1000	7.4908	6.0180	9.0409	998001	239467
Merge Sort	5000	0.4695	0.4600	0.4954	55197	61808
Heap Sort	5000	0.9678	0.8969	1.0771	118966	56982
Quick Sort	5000	3.0674	2.3008	3.6901	72958	31902
Selection Sort	5000	7.3589	4.2275	12.2109	12502500	4987
Insertion Sort	5000	20.4967	11.8128	41.9934	12497500	6304020
Bubble Sort	5000	37.6701	29.6755	62.0161	24990001	6179878
Quick Sort	10000	0.8499	0.7570	1.0290	173985	65005
Merge Sort	10000	1.6337	1.3475	2.0151	120525	133616
Heap Sort	10000	2.1137	1.3008	3.0915	258310	124154
Selection Sort	10000	22.9262	17.3337	31.3704	50005000	9987
Insertion Sort	10000	53.2889	47.0923	70.0495	49995000	24911684
Bubble Sort	10000	137.9329	131.0422	147.8324	99980001	24950202

Table 3: Benchmark Results – Sorted Input

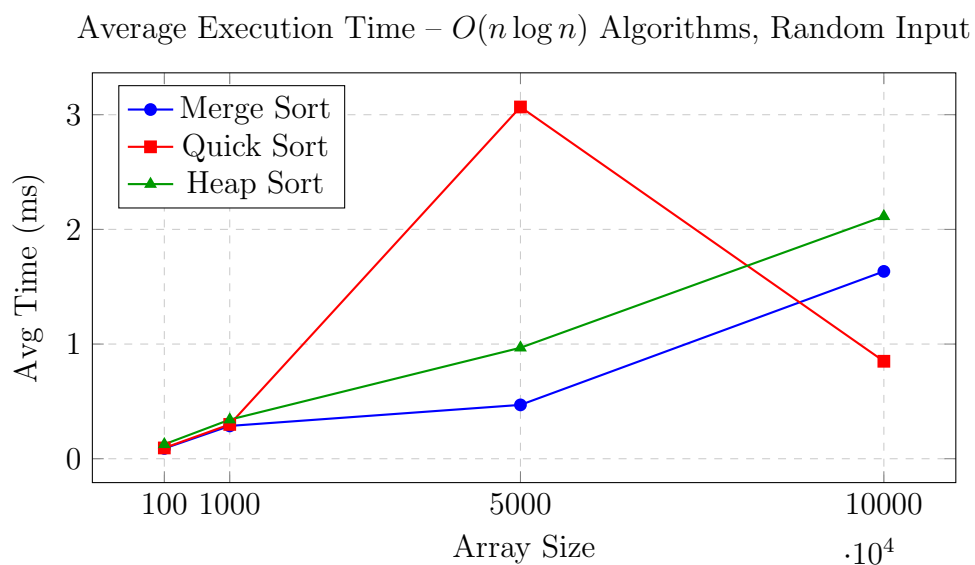
Algorithm	Size	Avg (ms)	Min (ms)	Max (ms)	Comparisons	Interchanges
Insertion Sort	100	0.0062	0.0054	0.0070	4950	0
Selection Sort	100	0.0050	0.0038	0.0083	5050	0
Bubble Sort	100	0.0109	0.0102	0.0119	9801	0
Merge Sort	100	0.0070	0.0052	0.0107	356	672
Heap Sort	100	0.0049	0.0038	0.0072	1384	641
Quick Sort	100	0.0044	0.0034	0.0058	646	367
Merge Sort	1000	0.0733	0.0667	0.0848	5044	9976
Quick Sort	1000	0.0587	0.0491	0.0750	10846	5723
Heap Sort	1000	0.0893	0.0822	0.1010	20420	9709
Selection Sort	1000	0.2323	0.1593	0.3346	500500	0
Insertion Sort	1000	0.4216	0.3257	0.6335	499500	0
Bubble Sort	1000	0.8169	0.5903	0.9752	998001	0
Quick Sort	5000	0.3114	0.2857	0.3452	72138	37903
Merge Sort	5000	0.4995	0.4413	0.6051	32004	61808
Heap Sort	5000	0.4991	0.4487	0.5163	126868	60933
Selection Sort	5000	5.3746	3.8251	7.2578	12502500	0
Insertion Sort	5000	11.2135	9.0433	14.3527	12497500	0
Bubble Sort	5000	18.7207	16.0537	23.8441	24990001	0
Quick Sort	10000	0.6790	0.6019	0.8616	154747	85144
Merge Sort	10000	0.9505	0.8142	1.2162	69008	133616
Heap Sort	10000	1.2204	0.9151	1.5714	273916	131957
Selection Sort	10000	18.9899	15.3067	30.8414	50005000	0
Insertion Sort	10000	41.7824	34.4684	51.8751	49995000	0
Bubble Sort	10000	69.9102	60.0801	82.5504	99980001	0

Table 4: Benchmark Results – Inversely Sorted Input

Algorithm	Size	Avg (ms)	Min (ms)	Max (ms)	Comparisons	Interchanges
Insertion Sort	100	0.0086	0.0081	0.0093	4950	4950
Selection Sort	100	0.0045	0.0039	0.0058	5050	50
Bubble Sort	100	0.0127	0.0114	0.0143	9801	4950
Merge Sort	100	0.0076	0.0060	0.0113	316	672
Quick Sort	100	0.0048	0.0045	0.0059	669	379
Heap Sort	100	0.0488	0.0126	0.0675	1136	517
Merge Sort	1000	0.0807	0.0771	0.0864	4932	9976
Quick Sort	1000	0.1176	0.0449	0.3956	11542	6465
Heap Sort	1000	0.1116	0.0698	0.1414	17636	8317
Selection Sort	1000	0.4188	0.4002	0.4564	500500	500
Insertion Sort	1000	0.9706	0.7502	1.6896	499500	499500
Bubble Sort	1000	1.0576	0.8914	1.1740	998001	499500
Quick Sort	5000	0.3602	0.3258	0.4310	76161	39570
Merge Sort	5000	0.4401	0.4065	0.4836	29804	61808
Heap Sort	5000	0.5563	0.3925	1.0729	111876	53437
Selection Sort	5000	10.6077	8.4801	12.1774	12502500	2500
Insertion Sort	5000	17.9388	14.8867	26.7696	12497500	12497500
Bubble Sort	5000	28.6597	20.2098	44.9517	24990001	12497500
Heap Sort	10000	0.6826	0.5386	0.8823	243396	116697
Quick Sort	10000	0.6953	0.6543	0.7469	159679	91048
Merge Sort	10000	0.8522	0.8082	0.9833	64608	133616
Selection Sort	10000	24.5932	22.3822	31.9379	50005000	5000
Insertion Sort	10000	62.0271	56.9301	77.8448	49995000	49995000
Bubble Sort	10000	90.9201	82.1334	108.7063	99980001	49995000

8.2 Timing Analysis – Average Time vs Array Size

8.2.1 $O(n \log n)$ Algorithms – Random Input

Figure 4: Average time for $O(n \log n)$ algorithms on random input

8.2.2 $O(n^2)$ Algorithms – Random Input

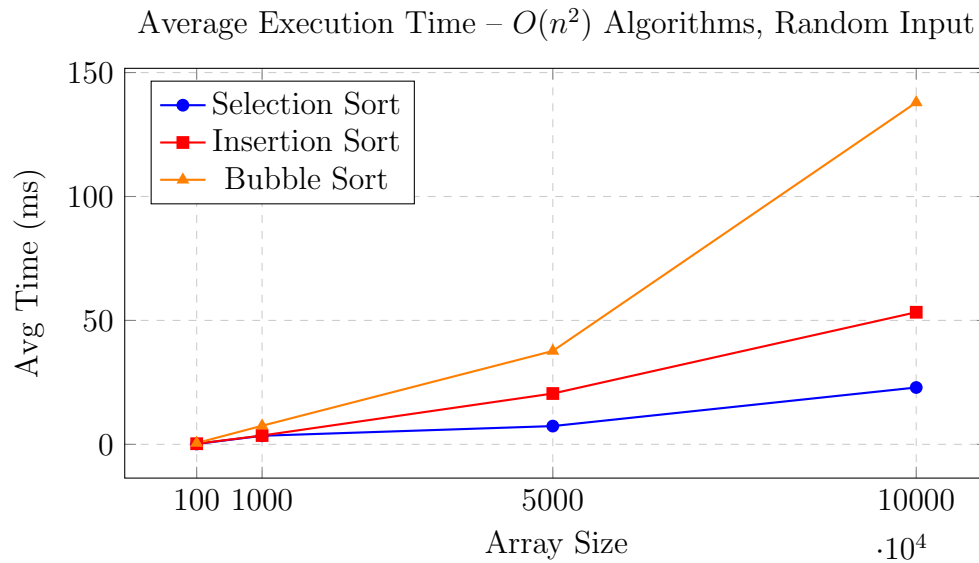


Figure 5: Average time for $O(n^2)$ algorithms on random input

8.2.3 All Algorithms at $n = 10000$ – All Input Types

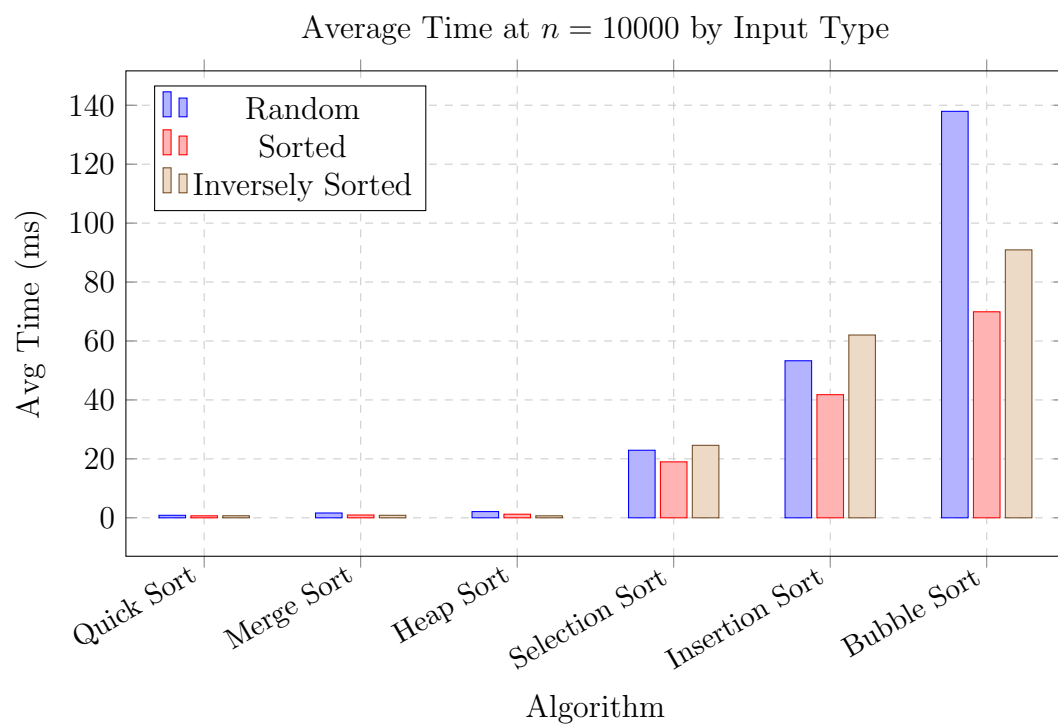


Figure 6: Average execution time at $n = 10000$ across all input types

8.3 Comparisons Analysis

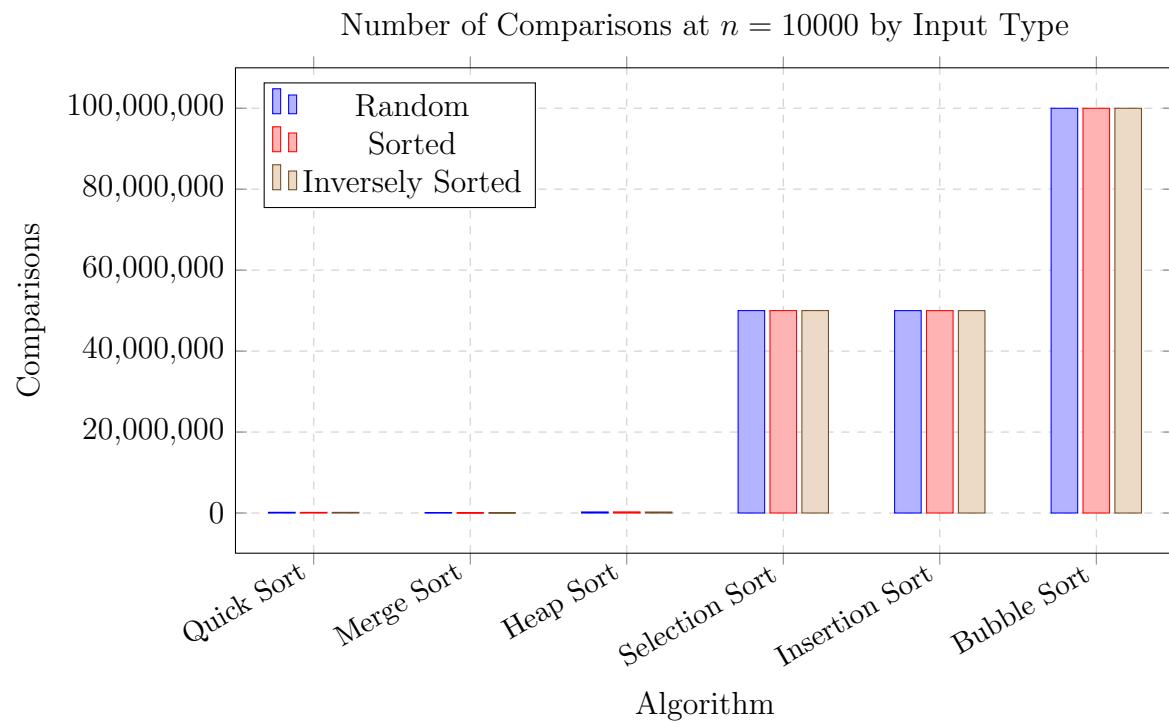


Figure 7: Total comparisons at $n = 10000$ across all input types